

界坞 WorldDock 产品需求文档 PRD

文档版本: v0.2

更新日期: 2026-05-24

主品牌: 界坞 WorldDock

社区模块: 界仓社区 WorldRepo / World Hub

产品类型: Agent 协作式世界推演工具 + 世界仓库社区

技术底座: 后端 Agent 能力基于 pi

一句话定位: 界坞是一个支持本地部署与网站托管的 Agent 协作式世界观创作平台。用户通过 Agent 推演世界设定、沉淀世界档案、孵化故事种子, 并可以像使用 GitHub 仓库一样将自己的世界发布到界仓社区, 被收藏、Fork、讨论和持续演化。

核心心智: 本地是 World IDE, 网站是 World GitHub。

品牌表达: 界坞 - 创造一个会长出故事的世界。

社区表达: 界仓 - 像仓库一样发布、收藏、Fork 和演化世界。

目录

1. 命名体系与品牌约束
2. 产品定位
3. 背景与问题
4. 产品目标与非目标
5. 用户角色
6. 核心产品原则
7. 核心概念定义
8. 产品形态与责任边界
9. 信息架构与路由
10. 核心页面需求
11. 界仓社区需求
12. 本地部署与 Push 需求
13. 网站计费需求
14. Agent 能力与 pi 技术底座
15. 上下文管理
16. 数据模型草案
17. API 草案
18. 权限、隐私与安全
19. 导入导出与世界包
20. 管理后台
21. MVP / P1 / P2 范围
22. 成功指标
23. 风险与应对
24. 验收标准
25. 发布路线建议
26. 最终产品边界

1. 命名体系与品牌约束

1.1 主品牌

产品主品牌定为：**界坞 WorldDock**。

命名含义：

- **界**：世界观、虚构世界、异世界、设定宇宙。
- **坞**：停泊、建造、维护、发布、集散，既有创作工坊感，也有仓库和社区感。
- **WorldDock**：对应“世界停泊、建造和发布的地方”，适合承载本地部署、网站托管和社区发布三种形态。

1.2 社区模块命名

网站社区模块命名为：**界仓社区**。

英文可用：**WorldRepo** 或 **World Hub**。

命名含义：

- “仓”对应 GitHub-like 的仓库心智。
- 每个公开世界都是一个“世界仓库”。
- 用户可以 Star、Fork、查看版本记录、发布更新，未来可以讨论和提交设定提案。

1.3 版本与模块命名

模块	中文名	英文名	说明
主产品	界坞	WorldDock	整体产品品牌
本地部署版	界坞 Local	WorldDock Local	本地部署创作工作台
网站托管版	界坞 Cloud	WorldDock Cloud	网站托管创作工作台
社区	界仓社区	WorldRepo / World Hub	公开世界仓库社区
创作工作台	世界工作台	World Workspace	Agent 协作推演核心界面
公开世界	世界仓库	World Repository	社区中的一个公开世界
世界数据包	世界包	World Package	导入导出和本地 Push 的数据包
故事入口	故事种子	Story Seed	由世界冲突触发的故事雏形

1.4 对外宣传语

首选：

界坞：创造一个会长出故事的世界。

备选：

- 界坞：创造、推演并发布你的世界。
- 界坞：从一个设定，长出整个世界。
- 界仓：像仓库一样维护你的世界。

1.5 命名使用准则

产品对外应避免使用以下表达作为主名称：

- AI 小说生成器
- AI Writer
- Novel Generator
- 一键写小说
- 自动写书工具

原因：这些表达会把用户心智引向“让 LLM 直接输出小说正文”，与界坞的核心定位不符。

推荐对外表达：

- Agent 协作式世界推演工具
- 世界观创作平台
- 世界仓库社区
- 故事种子孵化工具
- 可本地部署的世界工作台

2. 产品定位

2.1 核心定位

界坞不是 AI 小说生成器，也不是完整小说编辑器，而是：

世界推演 + 故事种子孵化 + 世界仓库社区。

用户不是来让 LLM 一键输出小说正文，而是与 Agent 一起：

1. 创建一个可持续生长的虚构世界；
2. 推敲世界规则、历史、势力、角色、冲突；
3. 从世界内部矛盾中孵化多个故事种子；
4. 将成熟的世界发布到界仓社区；
5. 允许他人浏览、Star、Fork、再创作或提出设定建议。

2.2 产品结构

界坞由两个核心部分组成：

World Workspace

Agent 协作式世界推演工作台

World Hub / WorldRepo

类 GitHub 的世界仓库社区，也就是界仓社区

2.3 产品心智

场景	心智	说明
本地部署	World IDE	用户用自己的模型 Key，本地创作和保存数据
网站托管	Cloud Workspace	用户无需部署，使用平台模型和云端工作台
社区	World GitHub	每个世界像仓库一样被发布、收藏、Fork 和更新

2.4 产品不是做什么

界坞不以下功能为核心：

- 生成完整小说正文；
- 管理章节；
- 排版发布；
- 连载小说；
- 一键生成百万字长篇；
- 替用户完成所有创作判断。

界坞真正提供的是：

灵感被放大
设定被推敲
冲突被发现
故事种子自然出现
世界被发布和演化

3. 背景与问题

3.1 用户问题

许多创作者在构思小说、游戏、TRPG、剧本或 IP 世界时，会遇到以下问题：

问题	表现
灵感碎片化	有一个有趣设定，但很难扩展成完整世界
设定不自洽	世界规则、社会后果、角色行为之间容易冲突

问题	表现
故事入口模糊	有世界观，但不知道哪些冲突可以发展成故事
AI 产物一次性	普通 LLM 对话难以沉淀成长期可维护的世界资产
社区协作弱	世界观很难像代码仓库一样被公开、Fork、持续演化
数据主权需求	一部分用户希望本地部署、自己配置模型、掌控数据

3.2 现有方案不足

现有方案	不足
通用 LLM 聊天工具	能对话，但缺少世界档案沉淀、版本、社区仓库结构
Scrivener / Notion / 飞书文档	能整理资料，但缺少 Agent 推演和故事种子孵化
AI 小说生成器	容易直接生成正文，无法支持世界长期演化
写作社区	主要承载作品，不承载“世界设定仓库”
GitHub	有仓库心智，但不适合非技术创作者直接维护世界观

3.3 产品机会

界坞的机会在于将三种能力组合起来：

- Agent 推演能力**：让模糊设定变成有后果、有矛盾、有故事入口的世界。
- 结构化沉淀能力**：把对话产物整理成可维护的世界档案。
- 社区仓库能力**：让世界像仓库一样被公开、Fork 和持续演化。

4. 产品目标与非目标

4.1 MVP 需要验证的四个闭环

4.1.1 创作闭环

用户能从一个模糊灵感出发，经过 Agent 协作，得到：

- 一个更完整的世界雏形；
- 一组结构化世界设定；
- 若干可继续发展的故事种子。

4.1.2 本地部署闭环

具备基础技术能力的用户可以：

- 通过 Docker Compose 部署界坞 Local；
- 配置自己的模型 API Key；

- 完整使用世界推演功能；
- 将数据保存在本地；
- 将确认公开的世界快照 Push 到界仓社区。

4.1.3 网站商业闭环

普通用户可以：

- 注册登录界坞 Cloud；
- 使用平台提供的模型；
- 无需配置 API Key；
- 按 token / 创作点消耗额度；
- 查看用量与余额。

4.1.4 社区发布闭环

用户可以：

- 将世界发布为公开世界仓库；
- 浏览他人公开世界；
- Star / 收藏世界；
- Fork 世界；
- 查看世界版本更新记录。

4.2 非目标

MVP 不做：

- 完整小说正文编辑器；
- 章节管理；
- 一键成长篇小说；
- 连载发布平台；
- 复杂多人实时协作；
- 真正 Git 协议兼容；
- 完整分支合并冲突解决；
- 任意模型市场；
- 社区内执行用户自定义代码；
- 网站版用户自定义模型 API Key。

5. 用户角色

5.1 普通创作者

特征：

- 想快速开始创作；
- 不想配置模型；
- 愿意使用界坞 Cloud；

- 关注灵感、世界观、故事种子；
- 可能愿意为用量付费。

核心需求：

- 打开网站即可创建世界；
- Agent 能帮助补全、推演、挑刺；
- 世界档案能自动沉淀；
- 故事种子有启发性；
- 可以公开自己的世界。

5.2 折腾型创作者 / 初级开发者

特征：

- 愿意部署 Docker；
- 有自己的模型 API Key；
- 关注数据控制；
- 希望本地使用，但也想发布到社区。

核心需求：

- 部署简单；
- API Key 配置清晰；
- 模型连接测试明确；
- 数据本地保存；
- 可将世界 Push 到界仓社区。

5.3 开发者 / 高级用户

特征：

- 熟悉部署；
- 可能希望自定义模型、Prompt、Agent 行为；
- 未来可能扩展本地版本。

核心需求：

- 可配置模型；
- 可导入导出；
- 可查看日志；
- 可理解上下文与 token 消耗；
- 未来支持高级自定义。

5.4 社区浏览者

特征：

- 不一定自己创作；
- 喜欢浏览世界观；
- 可能 Star、Fork 或参考他人世界。

核心需求：

- 发现有趣世界；
- 快速理解世界亮点；
- 查看设定、冲突、故事种子；
- Fork 到自己的空间继续创作。

5.5 平台管理员

核心需求：

- 管理用户；
- 查看 token 消耗；
- 控制平台成本；
- 审核社区内容；
- 处理举报；
- 下架违规世界；
- 管理模型配置。

6. 核心产品原则

6.1 永远先推演，再生成

Agent 的默认工作顺序是：

设定 -> 规则 -> 后果 -> 矛盾 -> 人物处境 -> 故事种子

Agent 不应优先输出小说正文。

6.2 对话负责生成，档案负责生长

产品不能只是 Chatbot。

每次有价值的对话都应该转化为：

- 世界规则；
- 势力；
- 角色；
- 历史事件；
- 冲突；
- 故事种子；
- 待定设定。

6.3 Agent 可以建议，但不能擅自改世界

Agent 可以生成归档建议，但正式写入世界档案必须经过用户确认。

6.4 本地与网站体验一致，责任边界不同

项目	界坞 Local	界坞 Cloud
创作体验	与网站尽量一致	与本地尽量一致
模型来源	用户自己的 API Key	平台统一提供
计费	用户自行承担模型服务商费用	平台按 token / 创作点计费
数据存储	用户本地或自有服务器	平台云端
社区能力	跳转网站使用	原生提供
主要价值	数据主权、可折腾、可私有化	免部署、云端、社区、商业体验

6.5 网站区别本地部署的最大价值是社区

界坞 Cloud 不是单纯的托管版，而是直接连接界仓社区。

社区能力包括：

- 公开世界；
- Star；
- Fork；
- 版本记录；
- Explore；
- 用户主页；
- 未来的讨论、提案、贡献者体系。

6.6 本地 Push 是公开发布，不是完整云同步

界坞 Local Push 到界仓社区时，默认只上传用户确认公开的世界快照。

默认不上传：

- 原始对话记录；
- 私密草稿；
- 模型配置；
- API Key；
- 本地日志；
- 未确认设定；
- 本地数据库完整备份。

7. 核心概念定义

7.1 世界 World

一个世界是界坞的基本创作单位。

包含：

- 世界名称；
- 一句话设定；
- 类型；
- 风格标签；
- 世界规则；
- 势力；
- 角色；
- 地点；
- 历史事件；
- 冲突池；
- 故事种子；
- 对话记录；
- 发布状态；
- 版本记录。

7.2 世界档案 Archive

世界档案是用户确认后的结构化设定集合。

包括：

- 世界总览；
- 世界规则；
- 势力；
- 角色；
- 地点；
- 历史事件；
- 资源；
- 禁忌；
- 冲突；
- 待定设定；
- 废弃设定。

7.3 故事种子 Story Seed

故事种子不是正文，也不是完整大纲，而是一个“叙事触发器”。

一个故事种子包含：

字段	说明
标题	种子名称
一句话钩子	快速理解故事吸引点
触发事件	故事为什么开始
核心冲突	主体矛盾

字段	说明
相关设定	依赖哪些世界规则
潜在主角	谁被冲突卷入
故事方向	悬疑、伦理、冒险、成长等
未解问题	留给用户继续推演
状态	未展开、可发展、高潜力、暂存、废弃

7.4 冲突 Conflict

冲突是故事种子的上游。

例如：

记忆是财产，还是人格的一部分？

冲突包含：

- 冲突标题；
- 冲突双方；
- 涉及规则；
- 涉及势力；
- 可能爆发事件；
- 可衍生故事种子。

7.5 世界仓库 World Repository

当一个世界被发布到界仓社区后，它成为一个世界仓库。

类似 GitHub 仓库，但内容不是代码，而是：

- 世界 README；
- 世界档案；
- 冲突池；
- 故事种子；
- 版本记录；
- Fork 关系；
- Star；
- 授权协议；
- 未来的讨论与设定提案。

7.6 世界包 World Package

界坞 Local Push 到界仓社区时上传的世界数据包。

它是公开发布快照，不是完整本地数据库备份。

建议结构：

```
manifest.json
world.json
archive.json
conflicts.json
story-seeds.json
relations.json
changelog.json
assets/
```

8. 产品形态与责任边界

8.1 界坞 Cloud

用户直接访问网站使用。

特征：

- 需要账号；
- 平台提供模型；
- 用户不可配置模型 API Key；
- 按 token / 创作点计费；
- 原生接入界仓社区；
- 云端保存世界；
- 支持公开发布；
- 支持 Fork 他人世界；
- 由平台负责模型、成本、安全、内容审核。

8.2 界坞 Local

用户自行部署。

目标用户：

- 有一定钻研能力的初学者；
- 熟悉 Docker / API Key / 环境变量的开发者；
- 重视隐私和数据主权的创作者。

特征：

- Docker Compose 部署；
- 用户配置自己的模型 API Key；
- 数据保存在本地；
- 不接入平台 token 计费；
- 可以导入导出；
- 可以连接社区账号；
- 可以 Push 世界到界仓社区；
- 社区浏览与互动跳转网站完成。

8.3 界仓社区

只存在于网站。

社区能力：

- 公开世界仓库；
 - Explore；
 - Trending；
 - 用户主页；
 - Star；
 - Fork；
 - 版本记录；
 - 授权协议；
 - 举报与审核；
 - P1 支持讨论与设定提案。
-

9. 信息架构与路由

9.1 网站公开区

```
/
  首页

/explore
  发现世界

/trending
  热门世界

/topics/:tag
  标签世界列表

/:username
  用户主页

/:username/:worldSlug
  公开世界仓库首页

/:username/:worldSlug/archive
  公开世界档案

/:username/:worldSlug/seeds
  公开故事种子

/:username/:worldSlug/conflicts
  公开冲突池

/:username/:worldSlug/forks
  Fork 列表

/:username/:worldSlug/releases
  版本记录

/:username/:worldSlug/discussions
  讨论区, P1

/:username/:worldSlug/proposals
  设定提案, P1
```

9.2 网站应用区

```
/app/worlds  
我的世界  
  
/app/worlds/new  
创建世界  
  
/app/worlds/:worldId/workbench  
世界工作台  
  
/app/worlds/:worldId/archive  
世界档案编辑  
  
/app/worlds/:worldId/seeds  
故事种子池  
  
/app/worlds/:worldId/conflicts  
冲突池, P1 可独立  
  
/app/worlds/:worldId/publish  
发布设置  
  
/app/worlds/:worldId/settings  
世界设置
```

9.3 网站账户区

```
/settings/profile  
个人资料  
  
/settings/billing  
用量与余额  
  
/settings/tokens  
Access Token 管理  
  
/settings/data  
数据导入导出  
  
/settings/security  
账号安全
```


9.4 本地部署版应用区

```
/setup
  本地初始化

/app/worlds
  我的世界

/app/worlds/new
  创建世界

/app/worlds/:worldId/workbench
  世界工作台

/app/worlds/:worldId/archive
  世界档案

/app/worlds/:worldId/seeds
  故事种子

/settings/model
  模型配置

/settings/data
  数据导入导出

/settings/community
  社区连接

/settings/system
  系统状态与日志
```

9.5 管理后台

```
/admin
  管理后台首页

/admin/users
  用户管理

/admin/worlds
  公开世界管理

/admin/reports
  举报处理

/admin/moderation
  内容审核

/admin/usage
  token 用量

/admin/costs
  成本看板

/admin/models
  平台模型配置

/admin/settings
  全局设置
```

10. 核心页面需求

10.1 世界列表页

目标：让用户快速进入已有世界或创建新世界。

每个世界卡片展示：

- 世界名称；
- 一句话设定；
- 类型标签；
- 风格标签；
- 世界设定数量；
- 故事种子数量；
- 最近更新时间；
- 发布状态；
- 本地版显示远程状态；
- 网站版显示公开状态。

状态设计：

运行形态	状态
界坞 Cloud	私有、未公开、已公开、有未发布更改
界坞 Local	本地、未连接社区、未发布、已发布、本地有未 Push 更改

P0 功能：

- 创建世界；
- 搜索世界；
- 打开世界工作台；
- 删除世界；
- 导入世界包；
- 网站版显示发布状态；
- 本地版显示社区连接状态。

10.2 创建世界页

目标：用轻量输入启动世界创作，而不是让用户填写复杂表单。

输入字段：

字段	必填	说明
初始灵感	是	用户的一句话或一段灵感
世界名称	否	可由 Agent 生成

字段	必填	说明
类型	否	奇幻、科幻、近未来、悬疑等
风格关键词	否	冷峻、史诗、荒诞、温暖等
不想要的方向	否	用于排除创作方向

流程：



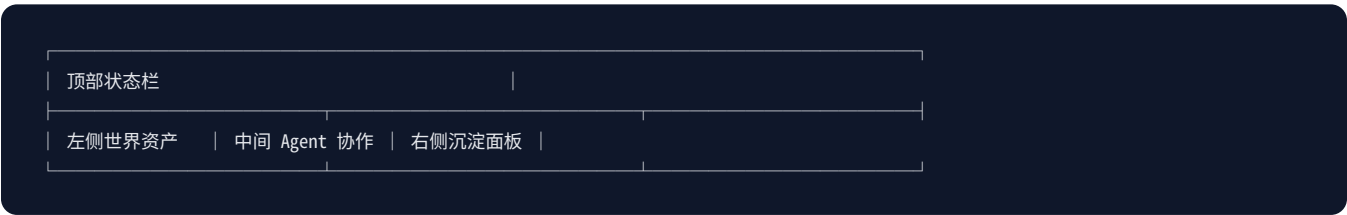
世界雏形确认卡包含：

- 世界标题建议；
- 一句话核心设定；
- 初始核心矛盾；
- 可推演方向；
- 风格判断；
- 建议的第一轮追问。

10.3 世界工作台

目标：这是界坞核心页面。用户在这里与 Agent 协作推演世界，并将有效内容沉淀成世界资产。

推荐布局：



顶部状态栏显示：

- 世界名称；
- 世界成熟度；
- 当前模式：Local / Cloud；
- 网站版：余额 / 本次预计消耗；
- 本地版：模型连接状态；
- 发布状态；
- 打开界仓社区按钮；
- 导出按钮。

左侧世界资产导航包含：

- 世界总览
- 世界规则
- 势力
- 角色
- 地点
- 历史事件
- 冲突池
- 待定设定
- 故事种子
- 最近对话

中间 Agent 协作区包含：

- 对话历史；
- Agent 模式按钮；
- 用户输入框；
- 流式输出；
- 工具调用状态；
- 本次 token / 创作点消耗。

Agent 模式按钮：

模式	作用
追问	针对模糊设定提出关键问题
扩展	把一个设定扩展成规则、后果、限制
挑刺	找漏洞、矛盾、不自洽之处
推演后果	推演设定对社会、制度、角色的影响
收束设定	整理为可保存设定卡
生成种子	从冲突中生成故事种子
检查一致性	与现有世界档案比对

右侧沉淀面板 Tab：

- 归档建议
- 一致性提醒
- 故事种子建议
- 当前上下文

归档建议

Agent 生成结构化卡片：

类型：世界规则
标题：记忆交易的基本限制
内容：记忆可以被提取、储存和转让，但出售者会永久失去该段记忆中的主观情感。
建议保存到：世界规则
操作：保存 / 修改后保存 / 丢弃

一致性提醒

用于提示新增设定和已有规则冲突。

示例：

你刚刚提出“记忆可以无限复制”。
这可能削弱已有设定“记忆稀缺性导致记忆金融体系成立”。
建议增加限制：复制记忆无法携带真实情感，或复制会损耗原记忆。

故事种子建议

用于将世界冲突转化成故事入口。

当前上下文

显示本轮 Agent 使用了哪些世界资产，帮助用户理解结果并控制 token 成本。

10.4 世界档案页

目标：查看、编辑、维护已确认的世界设定。

结构：

左侧：档案分类
中间：卡片列表
右侧：设定详情与关联内容

档案分类：

- 世界总览；
- 世界规则；
- 势力；
- 角色；
- 地点；
- 历史事件；
- 资源；
- 禁忌；
- 冲突；
- 待定设定；
- 废弃设定。

设定卡字段：

字段	说明
标题	设定名称
类型	规则 / 势力 / 角色等
摘要	简短说明
正文	详细设定
状态	已确认 / 待定 / 废弃
关联设定	与哪些设定有关
关联故事种子	哪些故事种子依赖该设定
创建来源	手动 / Agent 建议 / Fork 来源
更新时间	最近修改时间

10.5 故事种子池

目标：集中管理由世界矛盾生长出的故事入口。

故事种子卡片字段：

- 标题；
- 一句话钩子；
- 触发事件；
- 核心冲突；
- 涉及设定；
- 潜在主角；
- 故事方向；
- 未解问题；
- 状态；
- 潜力评分，P1；
- 来源冲突。

操作：

- 继续推演；
- 查看关联设定；
- 生成变体；
- 提升冲突强度；
- 保存为高潜力；
- 废弃；
- P1：试写场景。

重要限制：故事种子页不做章节管理、正文编辑器、一键写第一章、完整小说生成。

10.6 公开世界仓库页

路由：

/:username/:worldSlug

目标：让访问者快速理解一个公开世界，并决定是否 Star、Fork 或继续阅读。

页面结构：

顶部：世界名称、作者、Star、Fork、授权、更新时间
导航：Overview / Archive / Seeds / Conflicts / Releases / Forks
主体：世界 README
右侧：标签、统计、授权、作者信息

Overview 内容：

- 一句话设定；
- 世界类型；
- 风格标签；
- 核心规则；
- 核心矛盾；
- 推荐阅读路径；
- 高潜力故事种子；
- 最近更新；
- 授权方式；
- Fork 入口。

P0 操作：

- Star；
- Fork；
- 查看版本；
- 查看档案；
- 查看故事种子；
- 举报；
- 分享链接。

11. 界仓社区需求

11.1 Explore

目标：帮助用户发现公开世界。

筛选维度：

- 类型；
- 风格；
- 标签；
- 最近更新；

- Star 数；
- Fork 数；
- 故事种子数量；
- 世界成熟度。

P0 排序：

- 最新发布；
- 最近更新；
- 最多 Star；
- 最多 Fork。

P1 排序：

- Trending；
- 编辑推荐；
- 相似世界；
- 本周热门。

11.2 Star

目标：表达收藏和喜欢。

需求：

- 登录用户可 Star；
- Star 后进入个人 Starred Worlds；
- 世界页显示 Star 数；
- Star 不消耗 token。

11.3 Fork

目标：用户复制一个公开世界，发展自己的版本。

P0 行为：

```
点击 Fork
↓
复制公开世界档案、冲突池、故事种子
↓
生成自己的私有世界
↓
保留来源引用
```

Fork 后显示：

```
Forked from username/world-slug
```

P0 限制：

- 只 Fork 公开内容；
- 不复制原作者私密对话；
- 不复制原作者未发布草稿；

- 遵守原世界授权设置。

11.4 版本记录 Releases

目标：记录世界仓库每次公开更新。

每次发布记录包含：

- 版本号；
- 更新时间；
- 更新说明；
- 新增设定数量；
- 修改设定数量；
- 删除设定数量；
- 新增故事种子数量；
- 来源：界坞 Cloud Publish / 界坞 Local Push。

MVP 版本方式：不做真正 Git commit，仅做快照版本。

11.5 授权设置

每个公开世界必须设置授权方式。

P0 授权选项：

授权	说明
保留所有权利	仅可浏览，不允许 Fork
允许非商业再创作，需署名	可 Fork，不可商用
允许自由 Fork，需署名	可 Fork，可公开衍生
允许商业使用，需署名	可商用衍生
禁止 Fork，仅可浏览	不允许 Fork

公开世界页必须展示授权。

11.6 社区审核

P0 功能：

- 举报公开世界；
- 管理员隐藏世界；
- 管理员下架世界；
- 管理员封禁用户；
- 管理员查看举报列表；
- 管理员标记敏感内容；
- 管理员取消推荐或搜索收录。

P1 功能：

- 自动内容风险检测；

- 举报分类；
- 用户申诉；
- 敏感内容年龄提示；
- 社区规则页面。

12. 本地部署与 Push 需求

12.1 本地部署初始化

部署方式：MVP 优先支持 Docker Compose。

标准流程：

```
下载项目
复制 .env.example 为 .env
填写模型配置
docker compose up -d
访问 localhost
进入初始化页面
```

初始化页面步骤：

1. 检查数据库状态；
2. 配置模型；
3. 测试模型连接；
4. 创建本地管理员账户，可选；
5. 进入应用。

模型配置字段：

字段	说明
MODEL_PROVIDER	模型供应商
MODEL_BASE_URL	API 地址
MODEL_API_KEY	用户自己的 API Key
MODEL_NAME	模型名称
MAX_TOKENS	单次输出上限
TEMPERATURE	发散程度
CONTEXT_LIMIT	上下文限制

连接测试结果：

成功时显示：

- 模型连接成功；
- 当前模型名称；
- 上下文限制；
- 测试响应耗时。

失败时显示：

- API Key 无效；
- Base URL 不可达；
- 模型名不存在；
- 请求超时；
- 权限不足；
- 返回格式不兼容。

12.2 本地社区连接

目标：界坞 Local 用户可以连接网站账号，并将世界 Push 到界仓社区。

P0 连接方式：Personal Access Token。

流程：

```
graph TD; A[网站生成 Access Token] --> B[复制到界坞 Local]; B --> C[本地保存 Token]; C --> D[测试连接];
```

P1 支持网页登录授权。

本地社区状态：

```
graph TD; A[未连接社区] --> B[已连接账号]; B --> C[未发布]; C --> D[已发布]; D --> E[本地有未 Push 更改]; E --> F[远程版本已更新];
```

12.3 Push 到界仓社区

目标：用户将本地世界的公开快照发布到网站社区。

流程：

点击 Push 到社区
↓
选择创建新世界仓库或更新已有仓库
↓
选择公开范围
↓
选择要发布的内容
↓
预览差异
↓
填写更新说明
↓
确认 Push
↓
网站生成版本记录
↓
跳转公开世界页

默认发布内容：

- 世界总览；
- 已确认世界规则；
- 已确认势力；
- 已确认角色；
- 已确认冲突；
- 已确认故事种子；
- README；
- 标签；
- 授权设置。

默认不发布：

- 原始对话记录；
- 本地草稿；
- 未确认设定；
- 私密备注；
- 模型配置；
- API Key；
- 本地日志；
- token 记录。

差异预览 P0 做实体级 diff：

新增 3 项设定
修改 2 项设定
删除 1 项设定
新增 4 个故事种子

P1 再做字段级 diff。

13. 网站计费需求

13.1 计费原则

只有调用 Agent / 模型时才消耗额度。

不消耗额度的行为：

- 浏览社区；
- Star；
- Fork；
- 查看世界档案；
- 手动编辑设定；
- 导入导出；
- 发布世界；
- Push 世界，除非触发 Agent 生成发布说明；
- 删除世界。

消耗额度的行为：

- Agent 对话；
- 深度推演；
- 一致性检查；
- 故事种子生成；
- Agent 帮助生成 README；
- Agent 发布前风险检查；
- Agent 辅助审查提案，P1。

13.2 用户前台展示

界坞 Cloud 显示：

- 当前余额；
- 本月消耗；
- 本次预计消耗；
- 本次实际消耗；
- 最近用量记录；
- 余额不足提示。

建议前台使用“创作点”包装 token：

1 创作点 $\approx$ 1000 tokens

底层仍按真实 token 记录。

13.3 Usage Ledger

每次模型调用写入用量账本。

字段：

字段	说明
userId	用户
worldId	世界
conversationId	会话
agentRunId	Agent 调用
actionType	推演 / 检查 / 生成种子等
modelProvider	模型供应商
modelName	模型名称
inputTokens	输入 token
outputTokens	输出 token
cacheReadTokens	缓存读取 token
cacheWriteTokens	缓存写入 token
totalTokens	总 token
rawCost	模型原始成本
chargedCredits	扣除创作点
status	成功 / 失败 / 中断
createdAt	时间

13.4 扣费策略

推荐：

请求前：预估并冻结额度

请求后：按实际 usage 结算

失败时：释放冻结额度或按规则少量扣费

13.5 成本保护

P0 必须有：

- 单次最大 token；
- 单用户每日调用限制；
- 单世界上下文上限；
- 余额不足拦截；
- 超时中止；
- 管理员全局暂停开关；
- 异常消耗告警；

- 失败请求记录。

14. Agent 能力与 pi 技术底座

14.1 技术底座定位

后端 Agent 能力使用 **pi** 作为底座。

外部依赖：

```
https://github.com/earendil-works/pi
```

技术边界：

pi 负责 Agent Harness，界坞业务系统负责 World Platform。

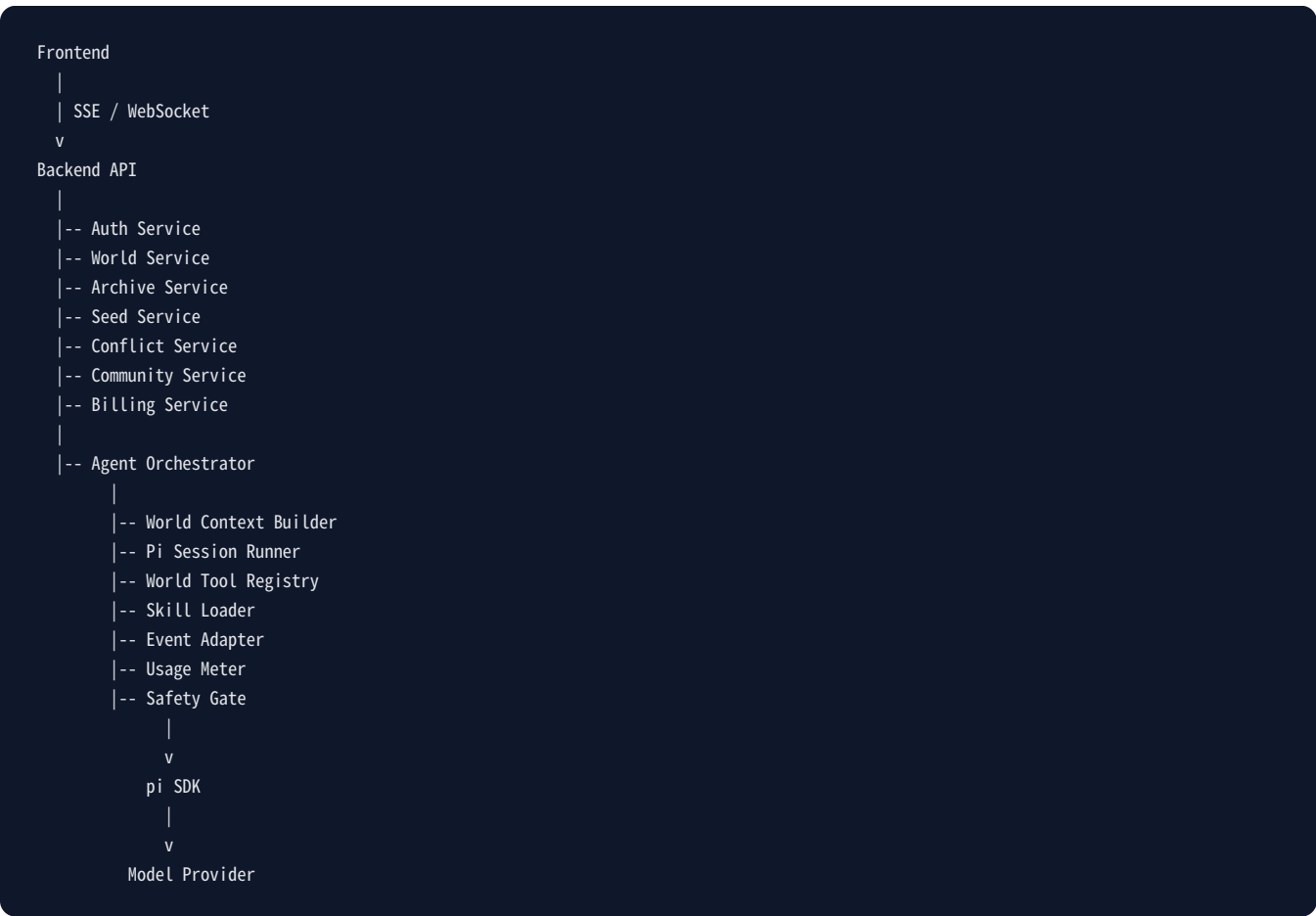
pi 负责：

- Agent session；
- LLM 调用；
- 流式事件；
- tool calling；
- Skills；
- Extensions；
- compaction；
- usage 获取。

界坞业务后端负责：

- 用户；
- 世界；
- 世界档案；
- 故事种子；
- 冲突；
- 社区仓库；
- Push / Publish；
- Fork；
- Star；
- 计费；
- 权限；
- 审核；
- 数据导入导出。

14.2 Agent 总体架构



14.3 Agent 模式

模式	用户心智	Agent 行为
追问	帮我问关键问题	针对模糊设定提出高质量问题
扩展	帮我扩成系统	从规则、代价、社会后果展开
挑刺	帮我找问题	找出逻辑漏洞、冲突、过强设定
推演后果	这个设定会造成什么	推演制度、阶级、宗教、经济影响
收束设定	帮我整理	输出可保存设定卡
生成种子	这里有什么故事	从冲突中生成故事种子
检查一致性	是否和已有设定冲突	检索世界档案并给出提醒
发布助手	帮我发布世界	生成 README、标签、更新说明，不直接发布

14.4 World Tools

Agent 只能调用受控的世界观领域工具。

只读工具：


```
world_get_overview
world_get_archive_summary
world_search_archive
world_get_related_entities
world_get_story_seed_summary
world_get_conflict_summary
world_get_recent_changes
```

建议型工具：

```
world_propose_archive_cards
world_propose_conflicts
world_propose_story_seeds
world_propose_consistency_warnings
world_propose_questions
world_propose_readme
world_propose_release_note
```

受控写入工具：

```
world_create_draft_suggestion
world_update_pending_suggestion
world_mark_suggestion_discarded
```

不允许 Agent 直接：

- 正式保存世界设定；
- 删除世界；
- 公开发布世界；
- Push 到社区；
- 扣费；
- 修改用户权限；
- 读取 API Key；
- 读取服务器文件；
- 执行系统命令。

14.5 安全限制

网站版必须禁用：

- bash；
- read 服务器文件；
- write / edit 文件；
- 任意网络请求；
- 任意用户自定义 extension；
- 读取环境变量；
- 读取 API Key；
- 执行社区世界中的代码或指令。

界坞 Local 普通模式同样默认禁用危险工具。P2 可以提供开发者模式，但必须明确风险。

14.6 Skills 设计

产品内置 Skills：

Skill	中文名	职责
world-architect	世界架构师	扩展规则、资源、制度、限制
world-critic	设定审校员	找漏洞、冲突、自治问题
world-simulator	历史推演者	推演设定对社会、经济、法律、宗教的影响
conflict-hunter	冲突猎人	从规则中提炼叙事矛盾
seed-incubator	种子孵化器	将矛盾转化为故事种子
publication-assistant	发布助手	生成 README、标签、更新说明、公开风险提示

网站版用户不可上传自定义 Skill。

本地版 P1 可允许用户添加本地 Skill，但默认关闭，并提示安全风险。

14.7 Extensions 使用边界

产品内置 Extensions：

Extension	作用
world-context-extension	注入本轮世界上下文
world-tool-extension	注册 World Tools
usage-meter-extension	汇总 token usage
safety-gate-extension	阻止危险工具和越权访问
world-compaction-extension	将长对话总结成世界推演摘要

限制：

- 网站版普通用户不允许安装自定义 Extension；
- 社区世界仓库不允许携带可执行 Extension；
- 本地版开发者模式可开放，但需明确风险；
- 所有 Extension 只能由可信代码包提供。

14.8 会话与 usage

产品使用原则：

- pi session 可用于 Agent 执行日志；
- 产品主数据以业务数据库为准；
- 世界档案不能依赖 pi session 才能恢复；
- 网站计费以后端读取的 usage 为准；

- 前端不能上报 token 作为结算依据。

15. 上下文管理

15.1 目标

避免每次调用都把整个世界塞进 prompt，控制成本并提高一致性。

15.2 World Context Builder

每次 Agent Run 前执行：

```
用户输入
↓
识别模式
↓
检索相关世界资产
↓
选择必要上下文
↓
注入 pi session
```

15.3 默认上下文内容

默认包含：

- 世界总览；
- 当前讨论对象；
- 相关规则；
- 相关冲突；
- 相关角色 / 势力；
- 最近变更；
- 当前 Agent 模式；
- 用户显式选中的上下文。

默认不包含：

- 全部对话历史；
- 全部世界档案；
- 全部故事种子；
- 全部社区评论；
- 全部 Fork 来源；
- 私密草稿。

15.4 前端显示

右侧“当前上下文”显示：

本轮已引用：

- 世界规则：记忆交易
- 势力：记忆银行
- 冲突：记忆是财产还是人格

用户可以手动添加或删除上下文。

16. 数据模型草案

16.1 User

```
User {  
  id  
  username  
  email  
  displayName  
  avatarUrl  
  role  
  createdAt  
  updatedAt  
}
```

16.2 World

```
World {  
  id  
  ownerId  
  title  
  slug  
  summary  
  type  
  styleTags[]  
  status  
  visibility  
  sourceType // cloud | local_import | fork  
  forkedFromWorldId?  
  maturityLevel  
  createdAt  
  updatedAt  
}
```

16.3 ArchiveItem

```
ArchiveItem {
  id
  worldId
  type // rule | faction | character | location | event | resource | taboo | note
  title
  summary
  content
  status // confirmed | pending | discarded
  source // manual | agent | fork | import
  relatedItemIds[]
  relatedSeedIds[]
  createdAt
  updatedAt
}
```

16.4 Conflict

```
Conflict {
  id
  worldId
  title
  description
  sideA
  sideB
  relatedArchiveItemIds[]
  potentialEvents[]
  status
  createdAt
  updatedAt
}
```

16.5 StorySeed

```
StorySeed {
  id
  worldId
  title
  hook
  triggerEvent
  coreConflict
  relatedArchiveItemIds[]
  relatedConflictIds[]
  potentialCharacters[]
  directionTags[]
  unresolvedQuestions[]
  status // raw | developable | high_potential | parked | discarded
  createdAt
  updatedAt
}
```

16.6 AgentRun

```
AgentRun {  
  id  
  userId  
  worldId  
  conversationId  
  mode  
  input  
  status  
  piSessionId?  
  modelProvider  
  modelName  
  inputTokens  
  outputTokens  
  totalTokens  
  cost  
  createdAt  
  completedAt  
}
```

16.7 Suggestion

```
Suggestion {  
  id  
  worldId  
  agentRunId  
  type // archive_card | conflict | story_seed | warning | question  
  payload  
  status // pending | accepted | edited_accepted | discarded  
  createdAt  
}
```

16.8 WorldRepository

```
WorldRepository {  
  id  
  worldId  
  ownerId  
  slug  
  visibility // public | unlisted | private  
  license  
  allowFork  
  allowProposal  
  starCount  
  forkCount  
  latestVersionId  
  createdAt  
  updatedAt  
}
```

16.9 WorldVersion

```
WorldVersion {  
  id  
  repositoryId  
  versionNumber  
  source // website_publish | local_push  
  changelog  
  snapshotUrl  
  addedCount  
  modifiedCount  
  deletedCount  
  createdAt  
}
```

16.10 Star

```
Star {  
  id  
  userId  
  repositoryId  
  createdAt  
}
```

16.11 Fork

```
Fork {  
  id  
  sourceRepositoryId  
  targetWorldId  
  targetRepositoryId?  
  userId  
  createdAt  
}
```

16.12 UsageLedger

```
UsageLedger {  
  id  
  userId  
  worldId  
  agentRunId  
  actionType  
  modelProvider  
  modelName  
  inputTokens  
  outputTokens  
  totalTokens  
  rawCost  
  chargedCredits  
  status  
  createdAt  
}
```

17. API 草案

17.1 World

```
GET    /api/worlds
POST   /api/worlds
GET    /api/worlds/:worldId
PATCH /api/worlds/:worldId
DELETE /api/worlds/:worldId
```

17.2 Archive

```
GET    /api/worlds/:worldId/archive
POST   /api/worlds/:worldId/archive
PATCH /api/archive-items/:itemId
DELETE /api/archive-items/:itemId
```

17.3 Story Seeds

```
GET    /api/worlds/:worldId/seeds
POST   /api/worlds/:worldId/seeds
PATCH /api/seeds/:seedId
DELETE /api/seeds/:seedId
```

17.4 Agent

```
POST   /api/worlds/:worldId/agent-runs
GET    /api/agent-runs/:agentRunId
POST   /api/suggestions/:suggestionId/accept
POST   /api/suggestions/:suggestionId/discard
GET    /api/agent-runs/:agentRunId/events
```

Agent 流式事件类型：

```
agent.run.started
agent.text.delta
agent.tool.started
agent.tool.finished
agent.suggestion.created
agent.warning.created
agent.usage.updated
agent.run.completed
agent.run.failed
```


17.5 Community

```
GET    /api/repositories/explore
GET    /api/repositories/:owner/:slug
POST   /api/repositories/:repoId/star
DELETE /api/repositories/:repoId/star
POST   /api/repositories/:repoId/fork
GET    /api/repositories/:repoId/releases
POST   /api/worlds/:worldId/publish
POST   /api/community/push
```

17.6 Local Push

```
POST /api/community/push
```

请求包含：

- manifest;
- world snapshot;
- archive;
- conflicts;
- story seeds;
- changelog;
- auth token。

18. 权限、隐私与安全

18.1 世界权限

行为	私有世界	公开世界
查看	owner	所有人
编辑	owner	owner
Fork	不可	取决于授权
Star	不可	登录用户
举报	不可	登录用户
Publish	owner	owner
Push	token owner	token owner

18.2 Agent 权限

Agent 只能访问：

- 当前用户有权限访问的世界；
- 当前世界相关数据；
- 用户显式 Fork 或导入的公开数据；
- 平台允许的工具。

Agent 不能访问：

- 其他用户私有世界；
- 模型 API Key；
- billing secrets；
- 系统环境变量；
- 管理后台数据；
- 未授权社区内容。

18.3 社区内容作为不可信上下文

社区世界、Fork 来源、评论、设定提案都可能包含 prompt injection。

规则：

- 社区内容不得进入 system prompt；
- 社区内容必须标记来源；
- 社区内容只能作为 user-level context；
- Agent 不得执行社区内容中的命令；
- 社区世界不得携带可执行 extension。

18.4 隐私默认值

默认不公开：

- 世界；
- 原始对话；
- 草稿；
- 未确认设定；
- API Key；
- 本地配置；
- 模型调用日志。

19. 导入导出与世界包

19.1 世界导出

支持导出：

world-package.zip

包含：

```
manifest.json
world.json
archive.json
conflicts.json
story-seeds.json
relations.json
metadata.json
```

manifest 示例：

```
{
  "schemaVersion": "1",
  "appVersion": "0.1.0",
  "source": "local",
  "exportedAt": "2026-05-24T00:00:00Z",
  "worldId": "local-world-id",
  "title": "失忆之城",
  "slug": "memory-city"
}
```

19.2 世界导入

导入时处理：

- schema 版本；
- 重复世界名称；
- ID 冲突；
- 缺失字段；
- 旧版迁移；
- 来源信息。

19.3 网站与本地互通

方向	需求
本地 -> 网站	本地可 Push 公开世界快照到界仓社区
网站 -> 本地	网站可导出 World Package，本地可导入
Fork -> 本地	P1 支持下载 Fork 后的世界包
本地备份	本地可导出完整可恢复数据包，但不等同于公开发布包

20. 管理后台

20.1 P0 功能

- 用户列表；
- 用户余额；
- 用户用量；
- 公开世界列表；
- 举报列表；
- 下架世界；
- 封禁用户；
- 今日 token 消耗；
- 今日成本估算；
- 失败请求列表；
- 全局暂停模型调用；
- 平台模型配置。

20.2 成本看板

显示：

- 今日调用次数；
- 今日 input tokens；
- 今日 output tokens；
- 今日成本；
- 成本最高用户；
- 成本最高世界；
- 模型失败率；
- 平均响应时间；
- 异常调用告警。

20.3 内容审核看板

显示：

- 待处理举报；
 - 被举报世界；
 - 被举报用户；
 - 已下架内容；
 - 搜索隐藏内容；
 - 申诉状态，P1。
-

21. MVP / P1 / P2 范围

21.1 P0 必须做

创作

- 创建世界；
- Agent 对话推演；
- Agent 模式按钮；
- 世界档案；
- 归档建议；
- 故事种子池；
- 一致性提醒；
- 上下文显示；
- 世界导入导出。

本地部署

- Docker Compose；
- `.env` 配置；
- 模型连接测试；
- 本地数据库；
- 本地世界创建与推演；
- 本地导出；
- 连接社区 Access Token；
- Push 世界到界仓社区。

网站托管

- 注册登录；
- 平台模型调用；
- token usage 记录；
- 余额 / 创作点；
- 余额不足拦截；
- 用量页面。

社区

- 用户主页；
- 公开世界仓库页；
- Explore；
- Star；
- Fork；
- Releases；
- 授权设置；
- 举报；
- 管理员下架。

Agent / pi

- pi SDK 集成；

- 禁用危险内置工具；
- World Tools；
- 内置 world skills；
- 流式事件；
- usage 汇总；
- 网站扣费；
- 本地 usage 展示。

21.2 P1 后续做

- 冲突池独立页面；
- 世界成熟度评分；
- 时间线；
- 势力关系图；
- 角色关系网；
- Discussions；
- 设定提案；
- Watch；
- Fork Graph；
- 发布前 Agent 风险检查；
- Agent 生成世界 README；
- 本地网页登录授权；
- 多模型配置；
- 本地自定义 Skills；
- 高级用量分析。

21.3 P2 远期做

- 真正分支 / 合并；
 - 多人协作；
 - 团队空间；
 - 世界模板市场；
 - 设定组件复用；
 - 商业授权；
 - 衍生作品关联；
 - 付费世界；
 - 企业私有化；
 - 本地模型支持；
 - 开发者插件生态。
-

22. 成功指标

22.1 创作指标

- 新建世界转化率；
- 首次 Agent 推演完成率；
- 归档建议保存率；
- 每个世界平均档案卡数量；
- 每个世界平均故事种子数量；
- 用户 7 日内继续编辑世界比例。

22.2 社区指标

- 公开世界数量；
- 每个公开世界平均 Star；
- Fork 率；
- Explore 点击率；
- Fork 后继续创作率；
- 本地 Push 数量；
- 本地 Push 后公开页访问量。

22.3 商业指标

- 网站注册转化率；
- 付费转化率；
- 人均 token 消耗；
- 毛利率；
- 余额不足后的充值率；
- 平均每次 Agent Run 成本。

22.4 本地部署指标

- Docker 部署成功率；
- 模型连接测试成功率；
- 本地活跃实例数量，若用户开启遥测；
- 本地 Push 到社区比例；
- 本地导出 / 导入使用率。

23. 风险与应对

23.1 产品跑偏成 AI 小说生成器

风险：用户期望一键生成小说正文。

应对：

- 核心按钮不叫“生成小说”；
- 主路径强调“推演、沉淀、种子”；
- 片段试写放 P1；
- 片段试写不能成为主按钮。

23.2 世界档案被 Agent 污染

风险：Agent 自动保存大量低质量设定。

应对：

- Agent 只生成建议；
- 用户确认后保存；
- 支持待定 / 废弃状态；
- 支持编辑后保存。

23.3 网站成本失控

风险：长上下文、多轮 Agent 调用导致模型成本高。

应对：

- 单次 token 上限；
- 余额冻结；
- 上下文选择；
- 当前上下文可视化；
- 管理员成本看板；
- 异常调用告警。

23.4 社区版权纠纷

风险：用户发布侵权世界或未经授权 Fork。

应对：

- 发布时选择授权；
- 举报机制；
- 管理员下架；
- Fork 遵循授权；
- 公开页展示授权；
- 社区规则明确禁止侵权内容。

23.5 Prompt Injection

风险：社区世界 README、评论、Fork 来源中包含恶意指令。

应对：

- 社区内容不进入 system prompt；
- 标记为不可信上下文；
- Agent safety gate；
- 禁止社区世界携带可执行扩展；

- 网站版禁用危险工具。

23.6 本地部署复杂度过高

风险：目标用户跑不起来。

应对：

- Docker Compose 优先；
- 初始化向导；
- 模型连接测试；
- 明确错误提示；
- `.env.example` 完整；
- 部署文档聚焦最短路径。

24. 验收标准

24.1 创作闭环验收

给定用户输入：

一个世界里，记忆可以被买卖。

系统应能完成：

1. 生成世界雏形；
2. Agent 提出关键追问；
3. 扩展出至少 3 条世界规则；
4. 提醒至少 1 个潜在设定冲突；
5. 生成至少 3 个故事种子；
6. 用户可以将设定保存进世界档案；
7. 用户可以继续推演某个故事种子。

24.2 本地部署闭环验收

用户应能：

1. 使用 Docker Compose 启动界坞 Local；
2. 配置模型 API Key；
3. 通过模型连接测试；
4. 创建世界；
5. 使用 Agent 推演；
6. 本地保存数据；
7. 连接社区账号；
8. Push 世界到界仓社区；
9. 在网站看到公开世界页。

24.3 网站计费闭环验收

用户应能：

1. 注册登录界坞 Cloud；
2. 获得初始额度；
3. 发起 Agent 推演；
4. 系统记录 input / output / total tokens；
5. 扣减创作点；
6. 前台显示本次消耗；
7. 余额不足时禁止继续调用模型。

24.4 社区闭环验收

用户应能：

1. 发布公开世界；
2. 在 Explore 中看到世界；
3. 访问公开世界仓库页；
4. Star 世界；
5. Fork 世界；
6. 查看版本记录；
7. 举报公开世界；
8. 管理员下架违规世界。

25. 发布路线建议

25.1 Alpha 阶段

目标：验证创作闭环和本地部署可用性。

范围：

- 界坞 Local；
- 基础 Agent 推演；
- 世界档案；
- 故事种子；
- 基础导入导出；
- 手动 Push 的雏形。

25.2 Beta 阶段

目标：验证网站托管和界仓社区。

范围：

- 界坞 Cloud；
- 用户登录；

- 平台模型；
- 创作点计费；
- 公开世界页；
- Explore；
- Star；
- Fork；
- Releases。

25.3 Public Launch

目标：形成社区网络效应。

范围：

- 社区发现能力增强；
- 内容审核完善；
- 本地 Push 流程打磨；
- 推荐机制；
- 官方示例世界；
- 官方世界模板。

26. 最终产品边界

界坞由两个核心部分组成：

World Workspace
Agent 协作式世界推演工作台

World Hub / 界仓社区
类 GitHub 的世界仓库社区

一句话边界：

界坞聚焦于“世界观推演、设定沉淀、故事种子孵化与世界仓库社区”。它支持网站托管和本地部署两种创作形态。界坞 Local 允许用户使用自己的模型 API Key，并可将确认公开的世界快照 Push 到界仓社区；界坞 Cloud 提供平台模型、token 计费、云端创作和社区网络效应。产品不以生成完整小说正文、章节编辑、排版发布或一键成书为核心目标。

核心判断标准：

用户不是来获得一篇小说。
用户是来创造一个会继续长出故事的世界。

附录 A：术语表

术语	定义
界坞	产品主品牌，英文 WorldDock
界坞 Local	本地部署版本
界坞 Cloud	网站托管版本
界仓社区	公开世界仓库社区
世界	用户创作与维护的虚构世界项目
世界档案	用户确认后的结构化设定集合
故事种子	从世界冲突中产生的故事触发器
冲突池	世界内部矛盾的集合，是故事种子的上游
世界仓库	公开发布到界仓社区的世界
World Package	导入导出或本地 Push 使用的世界数据包
Agent Run	一次 Agent 调用和执行过程
创作点	网站版对 token 消耗的产品化包装

附录 B：外部依赖

名称	用途	链接
pi	后端 Agent Runtime 技术底座	https://github.com/earendil-works/pi